Artificial Intelligence course

6th Semester

Bachelor in Informatics and Computer Engineering

Minimax algorithm

Nuno Leite

28 March 2022

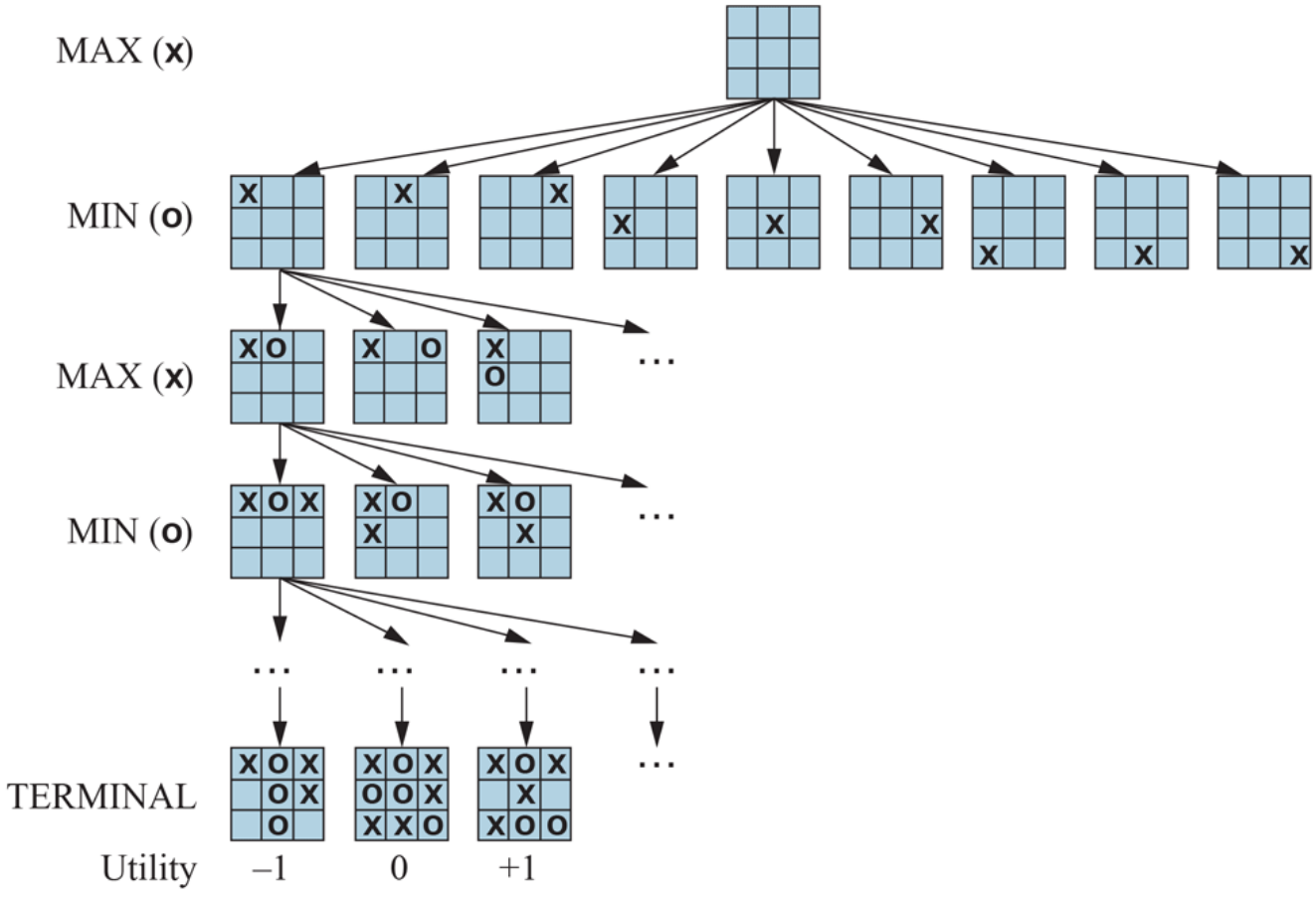
Two-player zero-sum games

- The games most commonly studied within AI (such as chess and Go) are what game theorists call deterministic, two-player, turn-taking, perfect information, zero-sum games
  - “Perfect information” is a synonym for “fully observable” and “zero-sum” means that what is good for one player is just as bad for the other: there is no “win-win” outcome
  - For games we often use the term **move** as a synonym for “action” and **position** as a synonym for “state”

Two-player zero-sum games

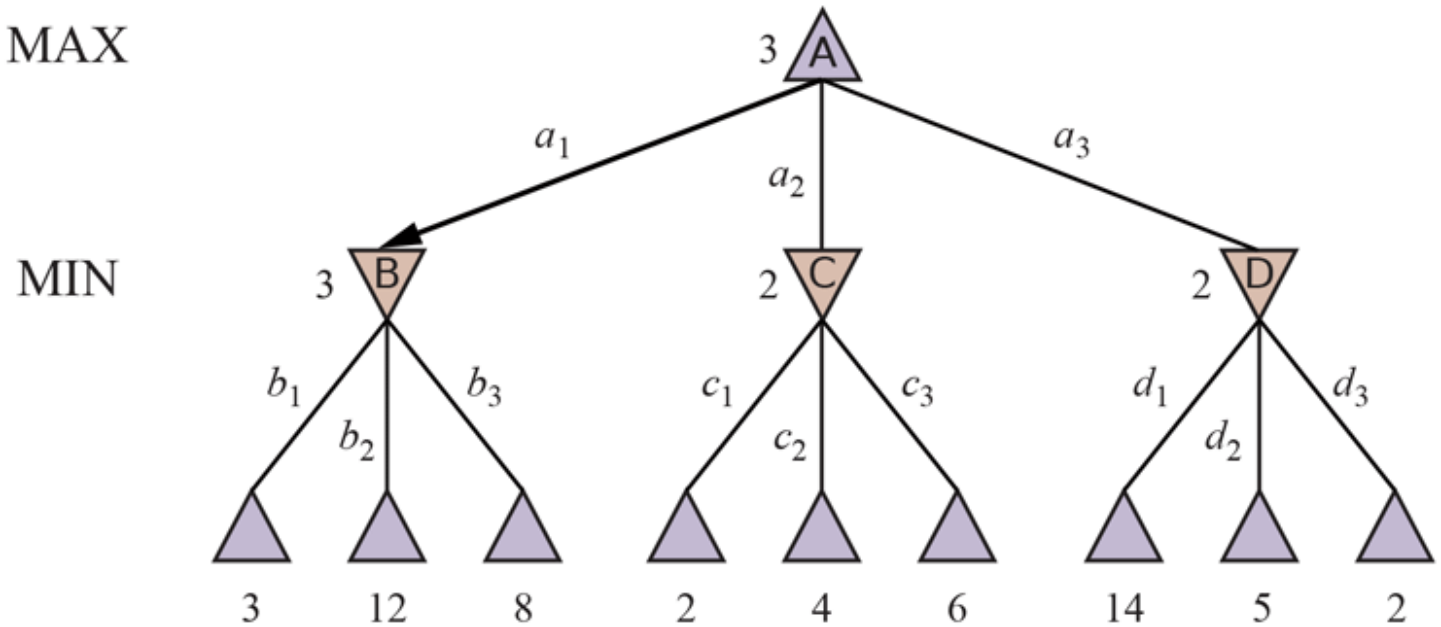
- We will call our two players MAX and MIN
- MAX moves first, and then the players take turns moving until the game is over
- At the end of the game, points are awarded to the winning player and penalties are given to the loser

Figure 5.1



A (partial) game tree for the game of tic-tac-toe. The top node is the initial state, and MAX moves first, placing an X in an empty square. We show part of the tree, giving alternating moves by MIN (O) and MAX (X), until we eventually reach terminal states, which can be assigned utilities according to the rules of the game.

Minimax tree



A two-ply game tree. The △ nodes are "MAX nodes," in which it is MAX 's turn to move, and the ▽ nodes are "MIN nodes." The terminal nodes show the utility values for MAX ; the other nodes are labeled with their minimax values. MAX 's best move at the root is a_1, because it leads to the state with the highest minimax value, and MIN 's best reply is b_1 , because it leads to the state with the lowest minimax value.

Minimax tree

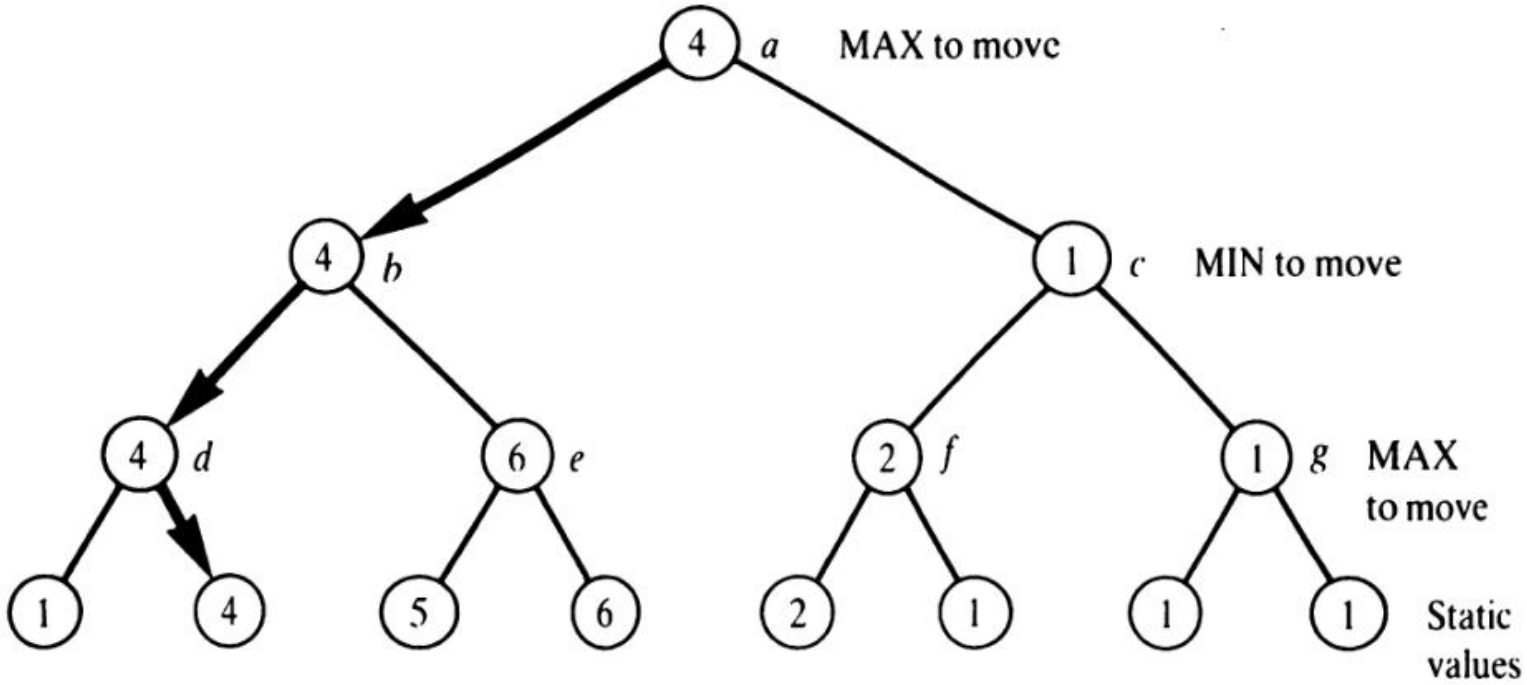


Figure 24.2 Static values (bottom level) and minimax backed-up values in a search tree. The indicated moves constitute the *main variation* – that is, the minimax optimal play for both sides.

Two-player zero-sum games

- A game can be formally defined with the following elements:
  - S_0: The initial state, which specifies how the game is set up at the start
  - TO-MOVE(s): The player whose turn it is to move in state s
  - ACTIONS(s): The set of legal moves in state s
  - RESULT(s, a): The transition model, which defines the state resulting from taking action a in state s
  - IS-TERMINAL(s): A terminal test, which is true when the game is over and false otherwise. States where the game has ended are called **terminal states**
  - UTILITY(s, p): A utility function (also called an objective function or payoff function), which defines the final numeric value to player p when the game ends in terminal state s

Example of utility functions

- In chess, the outcome is a win, loss, or draw, with values 1, 0, or $1/2$[2]
- Some games have a wider range of possible outcomes – for example, the payoffs in backgammon range from 0 to 192

The minimax value

- Given a game tree, the optimal strategy can be determined by working out the minimax value of each state in the tree, which we write as MINIMAX(s)
- The minimax value is the utility (for MAX) of being in that state, assuming that both players play optimally from there to the end of the game

The minimax value

- The minimax value of a terminal state is just its utility. In a non-terminal state, MAX prefers to move to a state of maximum value when it is MAX's turn to move, and MIN prefers a state of minimum value (that is, minimum value for MAX and thus maximum value for MIN). So we have:

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s, \text{MAX}) & \text{if IS-TERMINAL}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if TO-MOVE}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if TO-MOVE}(s) = \text{MIN} \end{cases}$$

The minimax search algorithm

Figure 5.3

```
function MINIMAX-SEARCH(game, state) returns an action
  player ← game.TO-MOVE(state)
  value, move ← MAX-VALUE(game, state)
  return move

function MAX-VALUE(game, state) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v ← −∞
  for each a in game.ACTIONS(state) do
    v2, a2 ← MIN-VALUE(game, game.RESULT(state, a))
    if v2 > v then
      v, move ← v2, a
  return v, move

function MIN-VALUE(game, state) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v ← +∞
  for each a in game.ACTIONS(state) do
    v2, a2 ← MAX-VALUE(game, game.RESULT(state, a))
    if v2 < v then
      v, move ← v2, a
  return v, move
```

An algorithm for calculating the optimal move using minimax—the move that leads to a terminal state with maximum utility, under the assumption that the opponent plays to minimize utility. The functions MAX-VALUE and MIN-VALUE go through the whole game tree, all the way to the leaves, to determine the backed-up value of a state and the move to get there.

Minimax implementation in Prolog

```
% minimax( Pos, BestSucc, Val):
%    Pos is a position, Val is its minimax value;
%    best move from Pos leads to position BestSucc

minimax( Pos, BestSucc, Val) :-
  moves( Pos, PosList), !,                    % Legal moves in Pos produce PosList
  best( PosList, BestSucc, Val)
  ;
  staticval( Pos, Val).                       % Pos has no successors: evaluate statically

best( [ Pos], Pos, Val) :-
  minimax( Pos, _, Val), !.

best( [Pos1 | PosList], BestPos, BestVal) :-
  minimax( Pos1, _, Val1),
  best( PosList, Pos2, Val2),
  betterof( Pos1, Val1, Pos2, Val2, BestPos, BestVal).

betterof( Pos0, Val0, Pos1, Val1, Pos0, Val0) :-    % Pos0 better than Pos1
  min_to_move( Pos0),                               % MIN to move in Pos0
  Val0 > Val1, !                                    % MAX prefers the greater value
  ;
  max_to_move( Pos0),                               % MAX to move in Pos0
  Val0 < Val1, !.                                   % MIN prefers the lesser value

betterof( Pos0, Val0, Pos1, Val1, Pos1, Val1).      % Otherwise Pos1 better than Pos0
```

Figure 24.3 A straightforward implementation of the minimax principle.

The alpha-beta algorithm

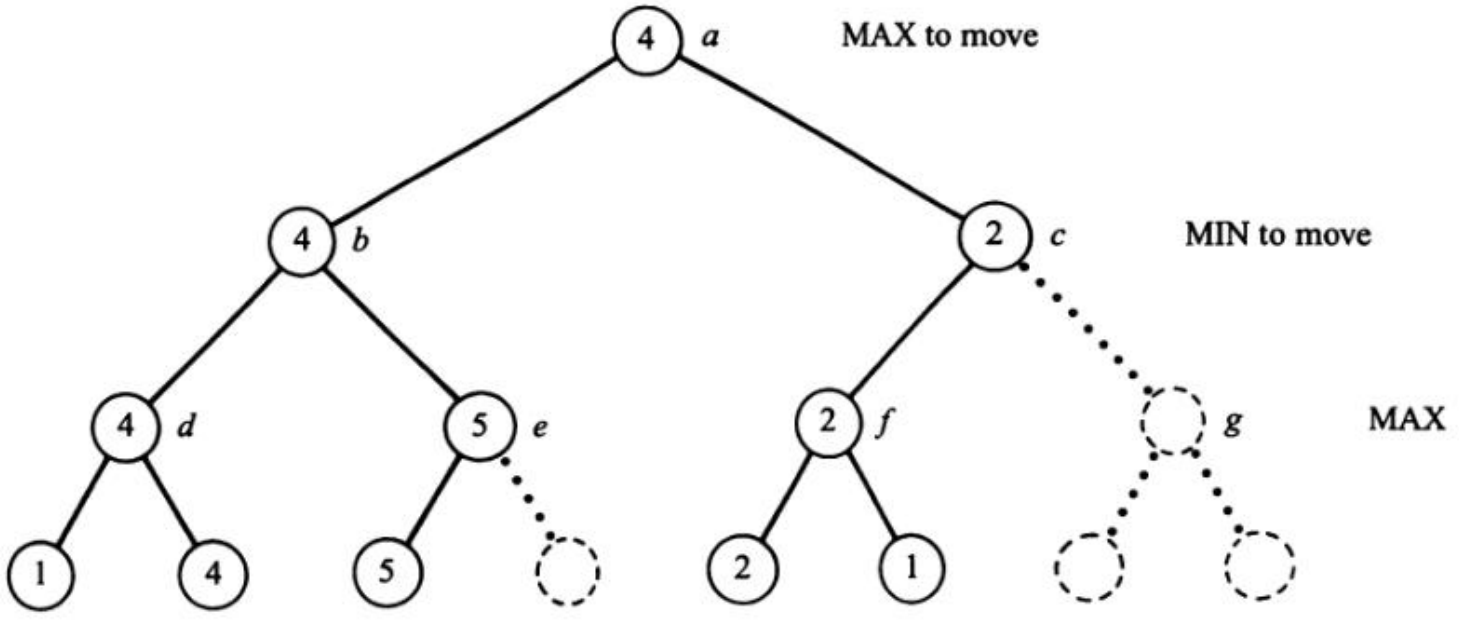


Figure 24.4 The tree of Figure 24.2 searched by the alpha-beta algorithm. The alpha-beta search prunes the nodes shown by dotted lines, thus economizing the search. As a result, some of the backed-up values are inexact (nodes *c*, *e*, and *f*; compare with Figure 24.2). However, these approximations suffice for determination of the root value and the main variation correctly.

The alpha-beta algorithm

```
% The alpha-beta algorithm

alphabeta( Pos, Alpha, Beta, GoodPos, Val) :-
  moves( Pos, PosList), !,
  boundedbest( Poslist, Alpha, Beta, GoodPos, Val);
  staticval( Pos, Val).                          % Static value of Pos

boundedbest( [Pos | PosList], Alpha, Beta, GoodPos, GoodVal) :-
  alphabeta( Pos, Alpha, Beta, _, Val),
  goodenough( PosList, Alpha, Beta, Pos, Val, GoodPos, GoodVal).

goodenough( [], _, _, Pos, Val, Pos, Val) :- !.      % No other candidate

goodenough( _, Alpha, Beta, Pos, Val, Pos, Val) :-
  min_to_move( Pos), Val > Beta, !;                  % Maximizer attained upper bound
  max_to_move( Pos), Val < Alpha, !.                 % Minimizer attained lower bound

goodenough( PosList, Alpha, Beta, Pos, Val, GoodPos, GoodVal) :-
  newbounds( Alpha, Beta, Pos, Val, NewAlpha, NewBeta),      % Refine bounds
  boundedbest( PosList, NewAlpha, NewBeta, Pos1, Val1),
  betterof( Pos, Val, Pos1, Val1, GoodPos, GoodVal).

newbounds( Alpha, Beta, Pos, Val, Val, Beta) :-
  min_to_move( Pos), Val > Alpha, !.                 % Maximizer increased lower bound

newbounds( Alpha, Beta, Pos, Val, Alpha, Val) :-
  max_to_move( Pos), Val < Beta, !.                  % Minimizer decreased upper bound

newbounds( Alpha, Beta, _, _, Alpha, Beta).          % Otherwise bounds unchanged

betterof( Pos, Val, Pos1, Val1, Pos, Val) :-         % Pos better than Pos1
  min_to_move( Pos), Val > Val1, !;
  max_to_move( Pos), Val < Val1, !.

betterof( _, _, Pos1, Val1, Pos1, Val1).             % Otherwise Pos1 better
```

Figure 24.5 An implementation of the alpha-beta algorithm.